PLATFORM OVERVIEW · TECHNICAL DUE DILIGENCE

Synora

The complete state of the platform

An evidence-based snapshot of every implemented subsystem, interface, and measurement — and an explicit account of what does not yet exist.

2,595	78,352	121	16
TESTS PASSING	**LINES OF SOURCE**	**DATA MODELS**	**AI ADAPTERS**
measured 2026-08-04	excluding tests	27 migrations	one interface

4 August 2026 · Repository state at commit **c2f3284**

Prepared for the founder, investors, enterprise customers, and technical due diligence

This document states what is not built as precisely as what is

Every figure was measured from the repository on the date above — the test totals come from suites executed against real PostgreSQL and Redis, not from projection. Capabilities that do not exist are listed by name in Part V and Appendix B, including capabilities that appear in internal directives. An interface is never presented as an implementation.

Current launch readiness, as the platform measures itself: **1 of 10 gates measured — not ready to launch.**

Document control

This document is a **snapshot, not a roadmap**. It describes Synora as the repository stands on 4 August 2026 at commit **c2f3284**. It was produced by inspecting the source tree, executing the test suites, and reading the accumulated engineering record — not by summarising intentions.

> **The rule this document is written under**
>
> Nothing here is invented, and nothing is estimated where it could be measured. An interface, an enum value, or a type declaration **is not an implementation**, and is never presented as one. Where a capability is absent it is stated as absent — including capabilities that appear in founder directives. Where something could not be verified in this environment it is marked **Not Yet Verified** rather than assumed.

How each claim was established

Claim type	Method of verification
Test results	Suites executed on 2026-08-04 against locally provisioned PostgreSQL 16 and Redis 7. Raw totals reported.
Counts (models, routes, adapters, components)	Enumerated directly from the source tree.
Design tokens	Read from `packages/design-system/src/styles/tokens.css`.
Subsystem status	Source inspection; a capability is "implemented" only where code executes and tests exist.
Launch readiness	Read from the platform's own gate board, which refuses to estimate.
Absent capabilities	Searched across the working tree and specification set; absence recorded explicitly.

Status vocabulary

Six categories are used, and they are not interchangeable:

Category	Meaning
A · WORKING NOW	Runs today with no credentials, no cloud, no deployment. Verified by executing tests.
B · NEEDS API KEYS	Complete implementation behind an adapter. Supplying a credential activates it — no code changes.
C · NEEDS CLOUD	Complete, but requires deployed infrastructure to exercise (managed Postgres, Redis, object storage).
D · PARTIAL	Real code exists and a named part is missing. The missing part is always named.
E · PLANNED	Committed in an existing written specification. Not built.
F · NOT IMPLEMENTED	No code and no specification. Directional intent only.

> **The most important distinction in this document**
>
> Categories B and C are **not** incomplete work. They are finished implementations waiting on a credential or a server. Category F is different in kind: it does not exist at all. Conflating the two is the single most common way platform documentation misleads a reader, and this document is structured specifically to prevent it.

Contents

Part I Company and product
- Executive summary
- Founder vision, mission, long-term vision
- Core philosophy
- Product positioning and the problems Synora solves
- Product overview

Part II Platform architecture
- Technology stack and system architecture
- Component topology and data flow
- User journey
- Workspace and multi-tenant architecture
- AI architecture and provider neutrality
- Memory architecture
- Plugin and extension architecture (status: absent)

Part III Security, operations and scale
- Security architecture
- Authentication and authorization
- Scalability, deployment and cost architecture
- Performance, observability, reliability and disaster recovery

Part IV AI quality and evaluation
- Model routing and provider orchestration
- Evaluation framework
- Benchmark framework
- Testing framework

Part V Capability register
- Every capability area in the founder directive, with honest status

Part VI Interface and design system
- Foundations, tokens, typography, theming
- Component system and application shell
- Every page and surface
- Keyboard, accessibility, motion, responsive behaviour

Part VII Feature inventory
- Categorised A through F, with evidence and risk

Part VIII Implementation matrix
- Module-by-module completion and test coverage

Part IX Quality evidence
- What was executed, what passed, what remains unproven

Part X Founder assessment
- What Synora is today
- Strengths, risks, and launch readiness

Part XI Investor view
- Differentiation, defensibility, economics, and what is not yet proven

Part XII Appendices
- Verification log · glossary · absent-capability register

PART I

Company and product

What Synora is, why it exists, and the principles that constrain every engineering decision recorded in this document.

Executive summary

Synora is an AI workspace: a multi-tenant, provider-neutral platform where teams hold conversations with AI models, organise that work into projects and folders, attach and search their own material, extend the system with tools, and see exactly what it costs.

The platform is substantially built. A production-grade backend of 43 route modules and 60+ services runs against a 121-model PostgreSQL schema with 27 migrations. Sixteen AI provider adapters sit behind a single interface. A complete authentication, authorization, and multi-tenancy system is enforced in the backend. A Next.js 15 front end delivers marketing, authentication, workspace, chat, knowledge, search, media, and administration surfaces. **2,595 automated tests pass.**

Three things distinguish it, and each is structural rather than aspirational:

- **Provider neutrality is architectural.** No business logic anywhere names a vendor. The platform runs on local models with no vendor account at all.
- **The platform refuses to fabricate.** An unavailable capability returns an honest error rather than plausible output. An unmeasured metric is reported as unmeasured, never as zero and never as passing.
- **Readiness is measured, not asserted.** An internal Engineering Operating System of 38 modules probes real code paths and reports evidence. It currently reports that Synora is **not ready to launch** — and that honesty is the feature.

> **The headline finding of this audit**
>
> The full backend suite — 2,098 tests across 209 files, including every database and queue integration test — was executed against real PostgreSQL 16 and Redis 7 during the preparation of this document. **All 2,098 passed. Zero failures.** This is the first complete execution of the integration suite since the project's CI provider became unavailable, and it substitutes for the automated verdict that has been missing.

What is **not** built is stated with equal precision. Autonomous AI agents, a canvas surface, a plugin system, a marketplace, MCP transport, a public SDK, mobile and desktop applications, offline mode, and a synchronisation engine **have no implementation in this repository**. They appear in founder directives; they appear in no specification and no source file. Part V records each one individually.

Founder vision

> *"Build Synora to become one of the world's most complete AI operating systems and AI workspaces — never by copying another company's product, but by studying published research, open standards, human-computer interaction, accessibility, and AI safety work, and then solving real user problems originally."*

That instruction has shaped the codebase in a specific and auditable way. Synora contains no reproduction of another company's proprietary system. Where the platform resembles the category it does so through convergent solutions to shared problems — conversations, branching, retrieval, cost accounting — implemented from first principles against public protocols and documented standards.

Mission

To give a team one workspace where AI work is organised, attributable, extensible, and economically legible — without binding that team to any single model vendor.

Each clause of that sentence maps to implemented architecture rather than intent. *Organised*: workspaces, projects, folders, conversation folders, tags, templates. *Attributable*: audit logs, activity records, per-execution tool audit trails, version history. *Extensible*: the universal tool engine with hardened HTTP transport. *Economically legible*: per-model pricing, credit ledger, usage records, and a cost monitor that detects anomalies. *Vendor-independent*: sixteen adapters behind one interface, including one that needs no account.

Long-term vision

Synora aims to become the layer where an organisation's AI work accumulates rather than evaporates: where a conversation becomes reusable knowledge, a failure becomes a permanent regression test, and a model change is a measurable decision instead of a leap of faith.

The mechanism for that already exists in a primitive form. Real interactions can be captured (consent-gated and redacted) into evaluation datasets; those datasets feed graders and a regression suite with persisted baselines; the results feed launch gates that refuse to estimate. This is the loop that turns operating experience into durable engineering evidence, and it is implemented today — though, honestly, it is exercised on very little real data so far.

Core philosophy

Five principles are enforced in code and visible in the commit history. They are the reason this document can be specific.

Principle	How it is enforced	Evidence
Honesty over appearance	Three states everywhere: passed / failed / unmeasured. Absence is never rendered as zero.	Launch gate board computes no overall percentage, by design.
Evidence over assertion	Readiness comes from probes against real code paths, never from self-assessment.	Privacy coverage probes real egress paths and has found two genuine leaks.
Capability absence over permission	Dangerous operations are made impossible rather than merely forbidden.	The automation execution layer has exactly four operations; merge and deploy do not exist in it.
Provider neutrality	Vendors sit behind adapters; business logic never names one.	16 adapters, one interface, one of which requires no credential.
Fail honestly, never plausibly	Unconfigured capabilities return errors; they never synthesise output.	Fabricated embeddings were found in an earlier version and deliberately removed.

Problems Synora solves

Problem	What Synora does about it	Status
AI work is scattered across browser tabs and personal accounts, leaving nothing an organisation owns.	Workspace-scoped conversations, projects, folders, tags, templates, and a searchable archive with audit trails.	WORKING NOW
Vendor lock-in makes switching models a rewrite and prices a hostage negotiation.	Sixteen adapters behind one interface; routing that recommends on evidence; a local-model path requiring no account.	WORKING NOW

Problem	What Synora does about it	Status
AI spend is invisible until the invoice arrives.	Per-model pricing with explicit basis, credit ledger, usage records, per-conversation cost, and anomaly detection.	WORKING NOW
Quality is unmeasurable, so teams cannot tell improvement from regression.	Evaluation datasets, graders, regression suite with persisted baselines, benchmark importer with holdout splits.	WORKING NOW
Customer data leaks into internal datasets and logs.	Redaction, consent gating, and an audit that probes real egress paths with known sensitive values.	WORKING NOW
AI systems overstate themselves, and the tooling around them inherits the habit.	Three-state reporting, no overall score, honest 503s, and gates that refuse to estimate.	WORKING NOW

Product positioning

Synora sits between two familiar categories and belongs to neither. A consumer chat product gives an individual a conversation but gives an organisation nothing durable. An enterprise AI gateway gives an organisation control but gives an individual no place to work. **Synora is the workspace where both hold: a surface a person actually uses, on top of infrastructure an organisation can audit, price, and switch.**

The technical positioning is sharper still, and it is unusual: Synora is a platform that measures itself and publishes what it does not know. That is a difficult thing to market and an easy thing to trust, which is the correct trade for enterprise and regulated buyers.

PART II

Platform architecture

The implemented system: its topology, its data, its flows, and the invariants that keep it coherent as it grows.

Technology stack

Layer	Technology	Notes
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS	Server Components by default; Client Components opt-in.
UI state	TanStack Query (server state), Zustand (UI state)	Separated deliberately.
Design system	In-repo package, 27 components, shadcn/ui-style	Token-driven; no raw colours in components.
Backend	Node.js 20+, Express, TypeScript (strict, ESM)	Layered: routes to services to data.
Database	PostgreSQL 16 with Prisma ORM	121 models, 27 migrations, drift-checked in CI.
Cache / queue	Redis 7 with BullMQ	10 workers. Used for queues today; caching is specified but not active.
Validation	Zod	Request validation and typed, stage-aware configuration.
Auth	Argon2, JWT access tokens, rotating refresh tokens	Custom implementation; session IDs and device metadata.
Vector	pgvector	Behind the embedding provider gate.
Realtime	Server-Sent Events (tokens), Socket.IO (presence)	Join authorization enforced at handshake.
Testing	Vitest, Supertest, Testing Library, Playwright (thin)	2,595 passing tests.
Build	pnpm workspaces, Turbo	Five packages.
Delivery	GitHub Actions, Docker	Four jobs: test, e2e, docker, security.

System architecture

Synora runtime topology
Every element verified present in the repository. Adapters are swappable; no business logic names a vendor.

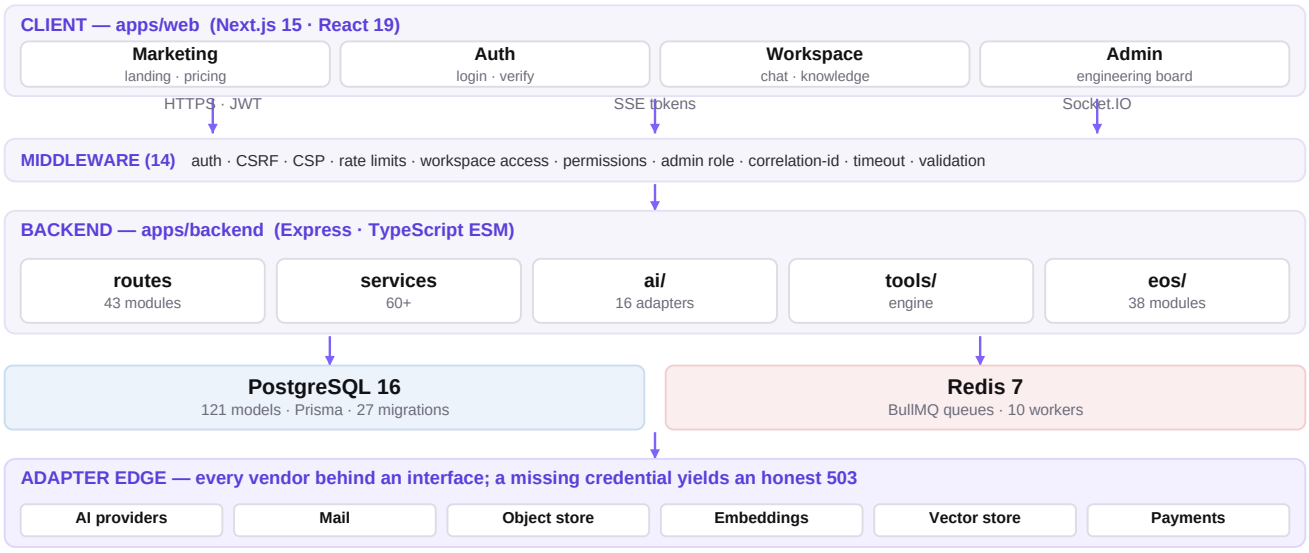


Figure 1 — Runtime topology. Every element shown was verified present in the repository.

Architectural invariants

Five rules hold across the codebase. A violation is treated as a defect, not a style preference, because each one protects a property that is expensive to recover once lost.

- **Routes to services to data.** Route modules contain no business logic.
- **One response envelope.** Every API response uses the shared `createApiResponse` / `createApiError` shape, so clients never special-case a subsystem.
- **Permissions are enforced in the backend**, never only in the interface.
- **Inaccessible resources return 404, not 403.** Existence is not disclosed to unauthorised callers.
- **Every vendor sits behind an adapter.** A missing credential yields an honest 503; it never yields invented output.

Data architecture

121 Prisma models across fourteen domains. The schema is the most complete artefact in the platform, and — stated plainly — it is **more complete than the product surface above it**. The feedback, evaluation, and model-registry clusters in particular have richer schemas than they have user-facing features.

Domain	Models	Representative
Identity and access	9	User, Account, Session, RefreshToken, ApiKey
Multi-factor auth	6	MFAMethod, MFAChallengeToken, MFARecoveryCode
Tenancy	9	Workspace, WorkspaceMember, Project, Folder, AuditLog
Conversation	12	Conversation, Message, MessageBranch, ConversationVersion
Memory	7	Memory, MemoryRelation, MemoryQuota, MemoryExport
Tools	12	ToolDefinition, ToolExecution, ToolSecret, ToolQuota
Search	6	SearchIndex, SavedSearch, SearchRanking
Feedback and learning	11	MessageFeedback, Correction, HallucinationReport
Evaluation	14	Dataset, EvaluationRun, BenchmarkRun, ABExperiment
Model registry	4	ModelRegistry, ModelVersion, TrainingCandidate

Domain	Models	Representative
Billing	10	Plan, Subscription, CreditLedgerEntry, UsageRecord
Knowledge	4	PromptTemplate, ConversationTemplate, Document
Media	3	GeneratedImage, GeneratedVideo, GeneratedAudio
Privacy	2	PrivacyConsent, TrainingConsent

Migration integrity is enforced, not assumed

A CI job asserts that applying the migration history reproduces `schema.prisma` exactly. This matters because a production deploy applies migrations — so the migration history, not the schema file, defines the production database. When the two diverge, production silently differs from every developer machine. An empty diff is the only passing result.

Data flow

Data flow — message to streamed response

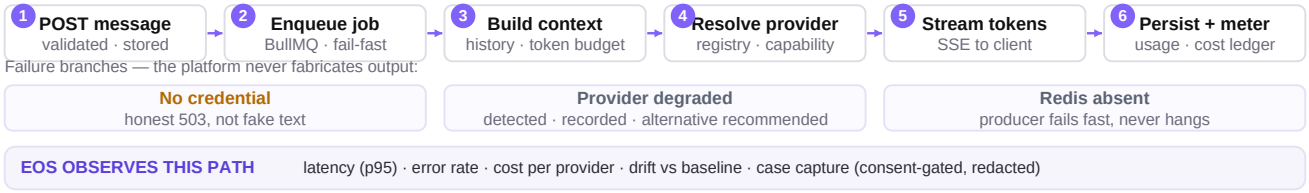


Figure 2 — The hot path, with its failure branches. The platform degrades honestly rather than plausibly.

User journey

Stage	What the user does	Implemented surface
Discover	Reads landing, pricing, features, changelog	Marketing routes, SEO artefacts (robots, sitemap)
Sign up	Registers, verifies email	Auth routes; email delivery needs a mail provider
Onboard	Lands in a workspace	Welcome route, workspace creation dialog
Work	Creates conversations, sends messages, sees streamed answers	Chat surface, SSE streaming, model picker
Organise	Files work into folders and projects, tags it, saves templates	Conversation folders, projects, tags, knowledge library
Explore	Branches an answer to try an alternative	Branching UI on the real branch API
Extend	Attaches files, registers a tool	File pipeline; tool engine with HTTP transport
Collaborate	Shares a conversation, sees presence	Token-scoped public sharing; Socket.IO presence and typing
Account	Reviews usage and cost, manages sessions and workspace	Analytics tables, settings surfaces
Govern	Reviews audit logs and readiness	Admin console, engineering readiness board

Workspace and multi-tenant architecture

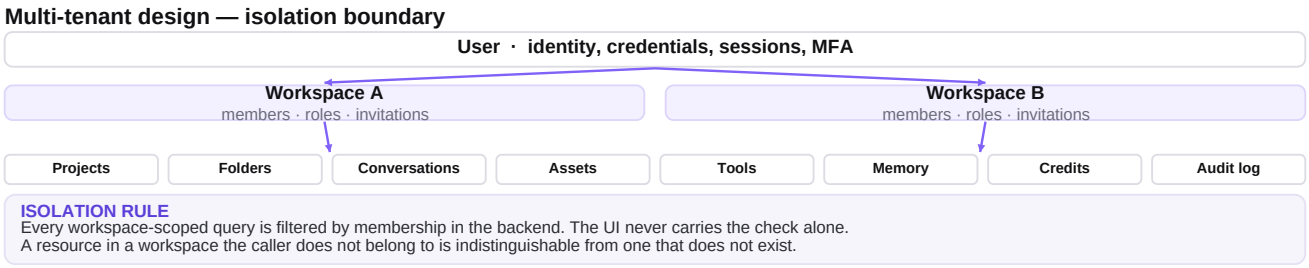


Figure 3 — Tenancy and the isolation boundary.

AI architecture

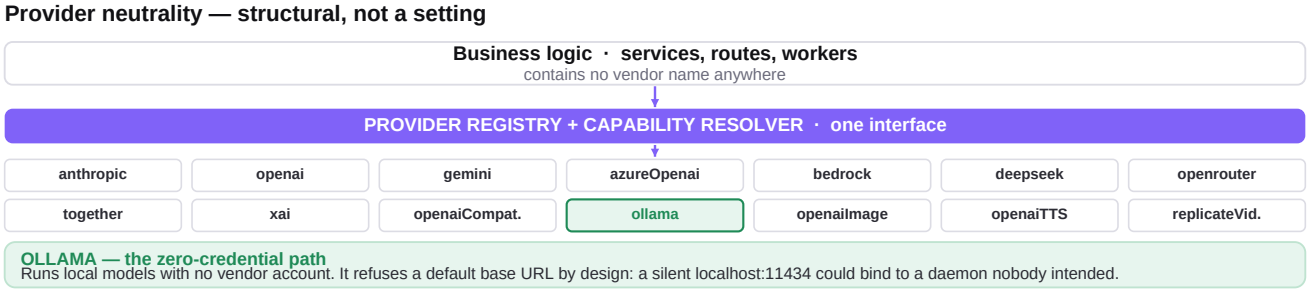


Figure 4 — Sixteen adapters behind one interface, and the credential-free path.

Beyond the adapters, the AI layer implements a model registry, capability resolution (matching a requested capability to a provider that offers it), streaming, and per-model pricing. The pricing layer carries an explicit `pricingBasis` field — token-priced versus per-image — because inferring the basis from the presence of a per-unit cost silently mislabels media generation, an error caught and corrected during development.

Memory architecture

Eleven memory services are implemented: retrieval, ranking, deduplication, summarisation, lifecycle, GDPR erasure, analytics, quota, cache, embedding, and relations. Seven data models support them.

> **The honest boundary of the memory system**
>
> Non-semantic memory operations work today. **Semantic retrieval requires an embedding provider and is gated behind it** — it returns an honest 503 rather than approximating. This is deliberate and hard-won: an earlier implementation fabricated embeddings, which produced confident nonsense, and it was removed rather than tuned.

Plugin and extension architecture

> **Status: no plugin system exists**
>
> There is no plugin architecture, no extension system, and no marketplace in this repository. What exists is the **universal tool engine**, which is a different and narrower thing: it registers and executes externally-hosted tools over HTTP. That engine is real, hardened, and described in Part V. A plugin system — third-party code running inside Synora with a lifecycle, a permission model, and a distribution channel — has neither implementation nor specification.

PART III

Security, operations and scale

How the platform protects data, who is allowed to do what, and what happens as load and failure arrive.

Security architecture

Control	Implementation	Status
Password storage	Argon2 hashing	WORKING NOW
Session tokens	JWT access tokens plus rotating refresh tokens carrying session ID and device metadata	WORKING NOW
Brute force	Account lockout and dedicated auth rate limiting	WORKING NOW
CSRF	Token-based protection on state-changing operations	WORKING NOW
Content security	CSP middleware with environment-aware sources	WORKING NOW
Authorization	Backend-enforced permissions, workspace access middleware, admin role gate	WORKING NOW
Resource abuse	Per-resource rate limiting	WORKING NOW
Input handling	Zod validation of body and query; request timeouts	WORKING NOW
Secret storage	SecretsManager with pluggable sources and audited reads; encrypted tool secrets	WORKING NOW
Outbound requests	SSRF-hardened HTTP transport (see below)	WORKING NOW
Auditability	AuditLog, ActivityLog, per-execution tool audit trail, MFA audit log	WORKING NOW
Supply chain	Blocking dependency audit (high+) and blocking full-history secret scan in CI	NEEDS CI
Multi-factor auth	Models, services and routes exist; not verified end-to-end	PARTIAL
Malware scanning	ClamAV provider abstraction with fail-closed policy	NEEDS SERVICE

The SSRF defence, in detail

Tool configurations are user-supplied, which makes the outbound HTTP transport an attack surface rather than a convenience. Four controls apply together: every address a hostname resolves to is validated **through the DNS lookup hook** — so a DNS answer cannot swap in a private address between the check and the connection; private-range IP literals are refused; redirects are not followed at all, because a redirect would bypass the original hostname's validation; and response size is capped.

Authentication and authorization

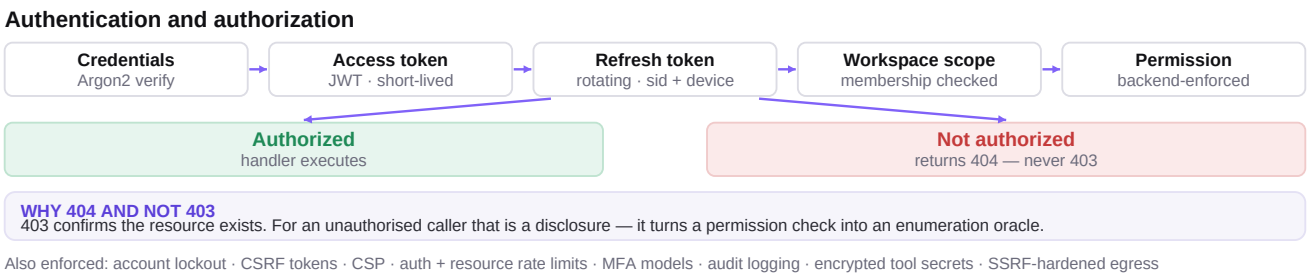


Figure 5 — The authentication chain and the disclosure-safe failure mode.

Scalability strategy

Dimension	Design	Verified?
Stateless API	Sessions live in signed tokens and the database, not process memory, so instances scale horizontally.	Design verified in source; horizontal scaling not load-tested.
Background work	Ten BullMQ workers consume queues independently of the request path.	Workers execute; throughput not measured.
Database	Indexed access paths, connection-pool tuning applied.	Present in schema and config; not benchmarked.
Streaming	SSE for tokens, Socket.IO for presence — separated so a slow stream cannot block presence.	Functionally verified by tests.
Caching	Redis provisioned and used for queues. **Not yet used as a cache.**	Specified (25-redis-cache.md); not implemented.

Scalability is designed for, not demonstrated

Every structural precondition for horizontal scale is present and inspectable. **No load test has been run.** Claims about concurrent user capacity would be fabrication, and none appear in this document. Operational simulation at 100 / 1,000 / 10,000 / 100,000 users is planned work, not completed work.

Deployment strategy

A production Dockerfile builds the backend image; CI builds that image on every change and runs a liveness smoke test against the running container, which catches the class of packaging bug a plain build would miss. Kubernetes manifests exist in the repository but are **not in use** — they belong to a later release. Deployment to a live environment has not been performed.

Cost architecture

Cost is treated as a first-class engineering concern with its own audit document (`docs/operations/COST_AUDIT.md`). Two findings from it are worth recording here.

Recurring cost today is one line item. Everything else — database, Redis, storage, email, observability, AI inference — runs locally at zero cost behind adapters, by design, until launch justifies paid infrastructure.

A measured 50% waste was found and eliminated. The CI pipeline was triggering twice per change (push and pull-request events overlapping on the same commits), doubling consumption at identical coverage. It was corrected and the fix verified live across four consecutive pushes.

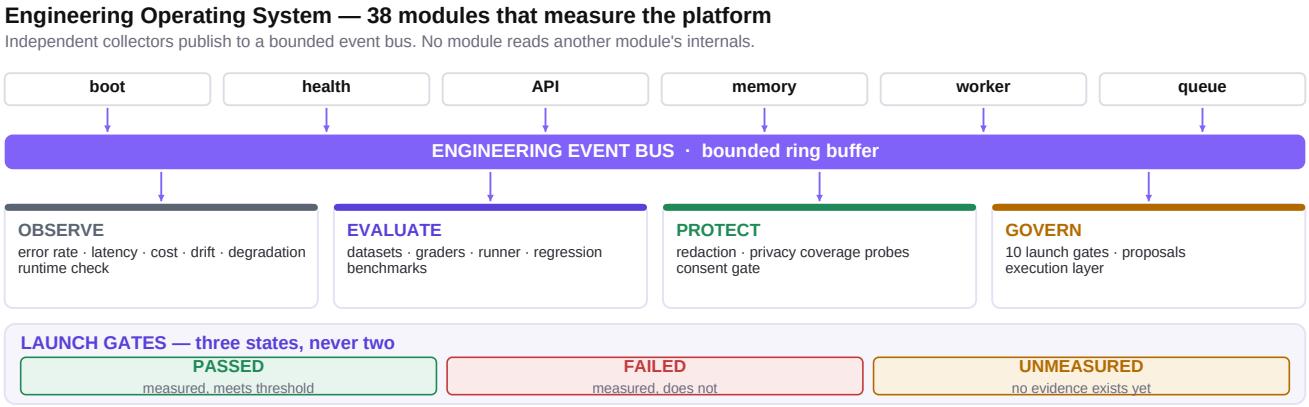
Optimisation	Assessment	Action
De-duplicate CI pipeline runs	No quality impact; pure waste	Implemented, verified
Cancel superseded PR runs	No quality impact	Implemented
Skip e2e/docker on PR pushes	Removes verification from commits that get merged	Rejected
Shallow-clone the security job	Blinds full-history secret scanning	Rejected
Drop the migration drift check	Protects production schema integrity; costs seconds	Rejected
Reduce coverage runs	Trades measurable quality for cost	Rejected on principle

Performance strategy

Implemented: database indexing, connection pooling, compression, pagination on list endpoints, code splitting and image optimisation on the front end, and latency instrumentation that reports **p95 rather than mean** — because the tail is what a customer actually experiences and the mean conceals it.

Not Yet Verified: no performance benchmark has been executed. Latency targets appear in the Release 3 specifications; no measurement against them exists.

Observability



Measured today: 1 of 10 privacy, at 100% across 2 declared egress paths. No overall percentage is computed — averaging unknowns manufactures knowledge.

Figure 6 — The Engineering Operating System: how evidence is produced.

Reliability and disaster recovery

Concern	Implemented	Gap
Provider failure	Degradation detection with alternative recommendation	Automatic failover not wired
Queue absence	Producers fail fast rather than hanging	—
Restart durability	Gate measurements and regression baselines persist and re-validate on load	—
Request stalls	Request timeout middleware	—
Tool failure	Retry with circuit breaker	—
Data loss	Disaster recovery documented (docs/DISASTER_RECOVERY.md)	**Never rehearsed.** No restore drill has been performed.
Fault injection	—	The recovery launch gate is unmeasured for this reason

PART IV

AI quality and evaluation

How Synora chooses a model, and how it establishes — with evidence — whether the answers are any good.

Model routing and provider orchestration

Routing is **evidence-driven and advisory**. The router scores providers on quality, cost, and latency, normalising all three identically so no dimension silently dominates, and reports whether its recommendation rests on measured evidence or on configuration. It flags close calls, because a relative score can make a two-millisecond difference look decisive.

> **The router recommends; it never intercepts**
>
> This is a deliberate constraint. A routing layer that silently redirects traffic is impossible to reason about when quality changes. Synora's router produces a recommendation with its basis attached, leaving the decision explicit and auditable.

Evaluation framework

Implemented: dataset management with origin labelling (real / synthetic / benchmark, required and never defaulted), pure graders, an execution runner, case capture from real traffic, a regression suite comparing against persisted baselines, and ten launch gates.

- **Case capture** turns real failures into permanent regression tests. It is consent-gated and redacted, and it stores the observed output separately from any reference answer — because storing a captured failure as the "expected" answer would invert every grader, scoring reproduction of a bug as success.
- **The regression suite compares against a stored baseline**, never a fresh copy of itself. Running a dataset twice and comparing proves nothing while turning the gate green; the first run captures a baseline and deliberately records no result.
- **The AI judge is not wired.** It needs a model to judge with. This is the direct reason three quality gates remain unmeasured.

Benchmark framework

A benchmark importer converts public benchmark datasets into executable evaluation datasets. Three properties are enforced by the code rather than by convention:

Property	Mechanism
Versioned	Every import records a content hash of exactly what was imported, alongside the declared source, pinned upstream version, and licence.
Reproducible	Re-importing identical content is an idempotent no-op. Importing different content under the same declared version is refused outright, naming both hashes.
Overfitting-resistant	A deterministic fraction of each import is diverted to a holdout split that routine gate measurement never executes. Tests assert zero holdout executions.

Status: the machinery is complete and tested; **no public benchmark dataset has been imported yet**, so the benchmark gate is correctly reported as unmeasured rather than as zero.

Testing framework

Verified test evidence

Executed 2026-08-04 against PostgreSQL 16 and Redis 7 provisioned locally. Not a projection.

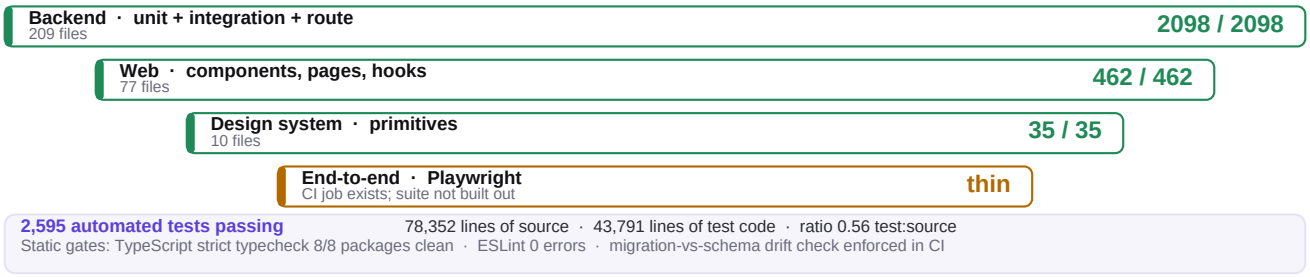


Figure 7 — Executed test evidence, measured during preparation of this document.

PART V

Capability register

Every capability area named in the founder directive, answered honestly — including, and especially, the ones that do not exist.

This part exists because a list of desired capabilities is not a list of built capabilities, and the difference is exactly what a reader of this document needs. Each row states what is actually in the repository.

Built and working

Capability	What exists	Status
AI Chat	Conversations, immutable messages, branching, streaming, model picker, attachments, editing, feedback capture	A
AI Workspace	Workspaces, members, invitations, projects, folders, tags, templates, audit trails	A
Knowledge	Prompt templates and conversation templates with a browsing and use surface	A
Search	PostgreSQL full-text search with a keyboard-driven dialog	A
Evaluation	Datasets, graders, runner, regression suite, benchmark importer, launch gates	A
Model routing	Evidence-driven advisory router with basis reporting	A
Provider orchestration	Registry, capability resolver, 16 adapters, pricing, degradation detection	A
Security	Full stack listed in Part III	A
Permissions	Backend-enforced, workspace-scoped, 404-not-403	A
Monitoring	EOS: 38 modules, event bus, collectors, drift and anomaly detection	A
Analytics	Usage summary and ledger with real aggregation	A
Admin console	Admin surface plus the engineering readiness board	A
Team collaboration	Sharing with scoped tokens, presence and typing over Socket.IO	A
Documents	Upload, processing pipeline, download, attachment to messages	A

Built, waiting only on a credential or a server

These are **complete implementations**. No business logic needs rewriting; supplying a credential or deploying infrastructure activates them.

Capability	What exists	What it waits for	Status
Vector memory	pgvector store, embedding adapters, retrieval and ranking services	An embedding provider credential	B
RAG	Retrieval, ranking, deduplication and summarisation over stored memory	Same embedding gate	B
Image	Provider abstraction, OpenAI adapter, byte storage, studio surface	An image provider credential	B
Voice	Text-to-speech adapter and audio route	A speech provider credential	B
Video	Async job abstraction, provider registry, polling API, worker	A video provider credential	B

Capability	What exists	What it waits for	Status
Billing	Plans, subscriptions, credits, ledger, invoices, refunds, webhooks with replay protection	Stripe credentials	B
Email delivery	Mailer abstraction and templates (invitations, verification, reset)	A mail provider credential	B
Malware scanning	ClamAV abstraction, three-state result, fail-closed policy	A ClamAV service	B
Object storage	Storage abstraction; local provider active	S3-compatible credentials	C
Enterprise readiness	Stage-aware config, enterprise and government mode hooks, secrets sources	Deployment and, for HSM, a real backend	C

Partially implemented — the missing part is named

Capability	What exists	What is missing	Status
Tool / plugin execution	Registration, execution, retry, circuit breaker, audit, quotas, encrypted secrets, HTTP transport	All other transports. GraphQL, Database, Search, File, Browser, Python and Custom are **enum values with no behaviour**	D
MCP	`MCP = 'MCP'` in a type union	Everything else. That line is the entire MCP presence in the repository	D
Developer platform	API key issuance and management	Portal, documentation site, usage dashboards, published SDK	D
Memory	Eleven services; non-semantic paths working	Semantic retrieval is gated on an embedding provider	D
MFA	Six models, services, routes	End-to-end verification	D
Browser tools	Playwright present in the CI toolchain	No browser-automation capability in the product	D
Accessibility	ARIA across 37 component files, full keyboard operation, visible focus	No audited WCAG conformance, no screen-reader pass, no automated a11y suite	D

Not implemented — no code, no specification

Read this section carefully

The capabilities below appear in founder directives. **None of them has any implementation in this repository, and none has a written specification.** They are recorded here so that no reader — founder, investor, or engineer — mistakes intent for progress. Every "agent" string match in the backend is HTTP `user-agent` header parsing.

Capability	Repository state	Status
AI Agents — autonomous multi-step execution	Absent. No agent loop, planner, or tool-use orchestration exists.	F
AI Canvas	Absent.	F
Automation engine	Absent.	F
Workflow engine	Absent. (Background workers are queue consumers, not a workflow engine.)	F
Plugin system	Absent. The tool engine is a different thing (see above).	F
Marketplace	Absent.	F
Extension system	Absent.	F

Capability	Repository state	Status
Public SDK	Absent. No published client library exists.	F
Mobile applications	Absent. The web app is responsive; no mobile app exists.	F
Desktop applications	Absent.	F
Offline architecture	Absent.	F
Synchronisation engine	Absent.	F

Planned — committed in a written specification, not built

Capability	Source specification	Current state
Elasticsearch-backed search	12-conversation-search.md	PostgreSQL FTS implemented instead
Collaborative message editing	07-collaboration.md	Presence and typing implemented; co-editing not
Version history interface	22-version-history.md	ConversationVersion model exists; no UI
Broader analytics dashboards	21-analytics.md	Usage analytics implemented; broader set partial
Redis as an active cache	25-redis-cache.md	Provisioned and used for queues only
End-to-end Playwright suite	RELEASE_2_PLAN.md (R2-h)	CI job exists; suite is thin
Remaining tool transports	16-tool-calling.md	Enum values only

PART VI

Interface and design system

Every surface that ships, the system it is built from, and an honest account of where interface quality has not yet been audited.

Foundations

Typography: Geist Sans for interface text and Geist Mono for code, loaded in the root layout and exposed as `--font-sans` and `--font-mono`.

Theming: dark-first. The root carries the dark palette and a light theme overrides it. All colours are HSL triplet custom properties consumed as `hsl(var(--token))`. **Components consume tokens and never raw colours** — a hex literal in a component is treated as a defect.

Design tokens — actual shipped values

Token	Dark	Light	Role
--background	240 10% 4%	0 0% 100%	Page surface
--foreground	240 6% 96%	240 10% 8%	Primary text
--card	240 8% 6%	0 0% 100%	Raised surface
--popover	240 8% 7%	0 0% 100%	Overlay surface
--primary	252 92% 68%	252 82% 58%	Brand violet, primary actions
--secondary / --muted	240 5% 12%	240 5% 96%	Secondary surfaces
--muted-foreground	240 5% 64%	240 4% 42%	Secondary text
--accent	240 5% 14%	240 5% 94%	Hover and accent surfaces
--destructive	0 72% 56%	0 72% 48%	Destructive actions
--success	152 60% 44%	152 60% 34%	Success states
--warning	38 92% 50%	32 95% 44%	Warning states
--border / --input	240 6% 14 / 16%	240 6% 90 / 88%	Edges and field borders
--ring	252 92% 68%	252 82% 58%	Focus ring
--radius	0.625rem	same	Corner radius base

Component system

27 design-system components in a shared package, all typed and tested:

- **Primitives:** Button, Input, Textarea, Checkbox, Switch, Label, Badge, Avatar, Separator, Kbd, Spinner, Skeleton
- **Containers and overlays:** Card, Container, Grid, Dialog, Tabs, DropdownMenu, Tooltip, Command, Breadcrumb, EmptyState, Toaster, Heading, CTA
- **Domain:** FeedbackCorrection and FeedbackHallucination — structured AI feedback capture wired to the feedback pipeline, not free text into a void

49 feature components in the application, grouped by domain: chat (composer, message bubble with attachments and editing, markdown with code highlighting, model picker, feedback controls), conversations (list, create dialog, folder tree), branching, search, files, sharing, collaboration, library, studio, history, analytics, settings, tools, auth,

and marketing.

Application shell

`AppShell` composes Sidebar, Topbar, WorkspaceSwitcher, UserMenu, NotificationsMenu, Breadcrumbs, CommandPalette, KeyboardShortcutsDialog, and CreateWorkspaceDialog. **Every shell component has a colocated test.**

Every page and surface

Navigation map — apps/web/src/app

PUBLIC

`(marketing)/`	landing · pricing · features · changelog
`invite/`	invitation acceptance
`share/`	shared conversation, token-scoped

AUTH

`auth/`	login · register · forgot · reset · verify-email
`welcome/`	post-signup

WORKSPACE (auth-guarded as a group)

`conversations/`	chat · list · detail · folders · branching
`knowledge/`	prompt and conversation templates
`search/`	full-text search
`studio/`	image · video · audio [credential-gated]
`developer/`	API keys [partial]
`admin/`	admin console · engineering readiness board

Global: error.tsx · global-error.tsx · not-found.tsx (all tested) · robots.ts · sitemap.ts

Figure 8 — Complete navigation map of shipped routes.

Keyboard shortcuts

Keys	Action	Scope
⌘ / Ctrl + K	Open command palette	Global
⌘ / Ctrl + /	Show keyboard shortcuts	Global
↑ ↓	Move through results	Palette, search dialog
↵ Enter	Select highlighted result / confirm	Palette, search, folder rename and create
Esc	Close dialog or menu	Global, search, folder tree

UX principles enforced in code

- **Honest empty states.** An absent baseline renders as "not established" — never as zero, because zero is a measurement and absence is not.
- **Unavailable is not broken.** Credential-gated features render as "not configured" rather than as errors, and never as fabricated output.
- **Loading is differentiated.** Skeletons where layout is known, spinners for actions.
- **Caveats carry equal visual weight.** On the readiness board, a proposal's caveats are rendered at the same weight as the proposal — an unmeasured draft must not look like a verified fix.

Accessibility

Present today	Not yet done
ARIA attributes and roles across 37 component files	No audited WCAG conformance statement
Semantic HTML and labelled inputs in primitives	No systematic screen-reader test pass
Full keyboard operation of palette, dialogs, search, folder tree	No automated accessibility suite (axe / Lighthouse) in CI
Visible focus rings via the --ring token	No formal contrast audit of every token pairing

Assessment: partially implemented. The foundations are real and the gaps are specific. They belong to the launch-validation phase and are recorded rather than glossed.

Motion and responsive behaviour

Motion — partial. Skeleton pulse, spinner rotation, and dialog/menu transitions from component defaults exist. **No documented motion language** — durations, easings, reduced-motion policy — has been defined. Nothing further is claimed.

Responsive — implemented, unevenly verified. Responsive utility classes are used throughout and the shell collapses for narrow viewports. **Not Yet Verified:** no systematic cross-breakpoint verification pass has been performed; that evidence is expected from the end-to-end suite, which is currently thin.

PART VII

Feature inventory

Module-level detail: what is finished, what remains, what it depends on, and what could go wrong.

Module	Finished	Remaining	Dependency	Risk	St.
Auth and identity	Registration, login, refresh rotation, logout-all, sessions, lockout, reset, verification	Email delivery; MFA end-to-end proof	Mail provider	MFA unverified end-to-end	A/D
Tenancy	Workspaces, members, roles, invitations, projects, folders, assets, audit logs	Invitation email send	Mail provider	Low	A
Conversations	CRUD, messages, branching, folders, tags, templates, versions, export/import	Version history UI	None	Low	A
Streaming	SSE token streaming, generation worker, streaming jobs	Load characteristics	Redis	Untested under concurrency	A
Realtime	Socket.IO presence and typing, handshake authorization	Collaborative editing	None	Scale unproven	A/E
AI providers	16 adapters, registry, resolver, pricing, streaming, degradation detection	Automatic failover	Credentials (except Ollama)	Low	A/B
Memory	11 services, 7 models, lifecycle, GDPR erasure, quotas	Semantic retrieval activation	Embedding provider	Quality unmeasured	D
Search	PostgreSQL FTS, dialog, indexes	Vector search activation; saved searches surface	Embedding provider	Scale unproven	A/D
Tools	Registration, execution, retry, breaker, audit, quotas, secrets, HTTP transport	Seven further transports incl. MCP	None	Enum implies more than exists	D
Files	Upload, processing pipeline, download, attachments	Cloud storage; virus scanning service	S3, ClamAV	Fail-closed policy untested live	A/B
Media	Image, video, audio abstractions, adapters, storage, studio UI	Provider activation	Credentials	Cost exposure at scale	B
Billing	Plans, subscriptions, credits, ledger, invoices, refunds, webhook dedup	Live Stripe verification	Stripe	Untested against real webhooks	B
Evaluation / EOS	38 modules, gates, regression, benchmarks, privacy probes, proposals	Judge wiring; datasets; 9 unmeasured gates	A judge model	Little real data yet	A/D
Admin	Admin console, engineering readiness board, audit surfaces	Broader analytics	None	Low	A

PART VIII

Implementation matrix

Completion and verification, module by module. No figure in this table is an estimate.

Subsystem	Implementation	Tests	Verified how	Blocking dependency
Auth and identity	Complete	Yes	Route + service suites vs live DB	Mail (email send only)
Tenancy and permissions	Complete	Yes	Route + permission suites	None
Conversations and messages	Complete	Yes	Route + service suites	None
Branching	Complete	Yes	API + UI component tests	None
Streaming (SSE)	Complete	Yes	Integration suite	None
Realtime (Socket.IO)	Complete	Yes	Integration suite	None
AI provider layer	Complete	Yes	Adapter + registry + pricing suites	Credentials to call live
Memory	Partial	Yes	Service suites; semantic path gated	Embedding provider
Search (FTS)	Complete	Yes	Route suites	None
Search (vector)	Complete, gated	Yes	Adapter tests	Embedding provider
Tool engine (HTTP)	Complete	Yes	Execution + SSRF tests	None
Tool engine (other transports)	Absent	n/a	Enum values only	Implementation
Files and processing	Complete	Yes	Pipeline + route suites	S3, ClamAV for production
Media (image/video/audio)	Complete, gated	Yes	Abstraction + route suites	Provider credentials
Billing	Complete, gated	Yes	Service + webhook suites	Stripe credentials
EOS (38 modules)	Complete	Yes	437 dedicated tests	Judge model for 3 gates
Design system	Complete	Yes	35 tests, 10 files	None
Web application	Complete	Yes	462 tests, 77 files	None
End-to-end suite	Thin	Minimal	CI job exists	Suite authoring
Agents / Canvas / Plugins / Marketplace / SDK / Mobile / Desktop / Offline / Sync	Absent	n/a	Searched; not present	Specification, then implementation

PART IX

Quality evidence

What was actually executed in preparing this document, what passed, and what remains unproven.

Executed during this audit

PostgreSQL 16 and Redis 7 were provisioned locally, all 27 migrations applied cleanly, and the complete suites executed on 4 August 2026:

Suite	Files	Tests	Result
Backend — unit, service, route, integration	209	2,098	**2,098 passed, 0 failed**
Web — components, pages, hooks	77	462	**462 passed, 0 failed**
Design system	10	35	**35 passed, 0 failed**
Total	**296**	**2,595**	**All passing**

Why this run matters more than a routine one

The project's CI provider has been unavailable for over a day — every job fails in seconds with no runner assigned, an infrastructure and billing condition unrelated to the code. That meant the database and queue integration tests — the majority of the backend suite — had gone unverified anywhere. Provisioning real infrastructure and executing them produced the verdict CI could not: **every one of the 2,098 backend tests passes.**

Static and structural gates

Gate	Result
TypeScript strict typecheck	8 of 8 packages clean
ESLint	0 errors (216 warnings, none blocking)
Migration-vs-schema drift	Enforced in CI; empty diff required
Dependency audit (high and critical, production)	Blocking gate in CI
Secret scan (full git history, gitleaks)	Blocking gate in CI
Docker production image	Built and liveness smoke-tested in CI
Test-to-source ratio	43,791 test lines to 78,352 source lines (0.56)

What is not proven

Stated plainly, because a quality section that only lists successes is marketing:

- **No load or performance test has ever been run.** No claim about concurrent capacity or latency under load appears anywhere in this document.
- **No disaster-recovery drill has been rehearsed.** The procedure is documented and untested.
- **The end-to-end suite is thin.** Full user journeys are not covered by automated browser tests.
- **Nine of ten launch gates are unmeasured**, including AI quality, hallucination rate, security testing, and recovery.
- **No live provider call has been made** — every AI path is verified against adapters and mocks, never a production endpoint.

- **Accessibility conformance is unaudited**, and no automated accessibility gate runs.
- **CI has been unable to run for over a day**, so no independent verification exists for the most recent commits.

Launch readiness, as the platform reports it

Launch gate board — as measured, 2026-08-04

GATE	MEASURED	THRESHOLD	STATE
Privacy tests 2 declared egress paths, probed	**100%**	100%	PASSED
AI quality needs a configured judge model	—	>=95%	UNMEASURED
Benchmark pass rate importer built; no dataset imported yet	—	>=98%	UNMEASURED
Hallucination rate needs factual-accuracy dataset	—	<2%	UNMEASURED
Memory accuracy needs multi-turn memory cases	—	>=95%	UNMEASURED
Security tests needs adversarial suite	—	100%	UNMEASURED
Regression failures needs a second run to compare	—	0	UNMEASURED
Integration tests suite passes locally; not yet gate-recorded	—	>=99%	UNMEASURED
Recovery success needs fault-injection runs	—	>=99%	UNMEASURED
Critical bugs needs triage feed	—	0	UNMEASURED

NOT READY TO LAUNCH — 1 of 10 gates measured
An unmeasured gate is not a failing gate. It is an absence of evidence, and it is reported as neither pass nor fail.

Figure 9 — The launch gate board, unedited.

PART X

Founder assessment

A direct account of what has been built, what it is worth, what is genuinely at risk, and what remains between here and launch.

What Synora already is

Synora is a **working, tested, multi-tenant AI workspace** with an unusually disciplined engineering substrate. A person can sign up, join a workspace, hold a conversation with an AI model, branch it, organise it, attach files, search it, share it, extend it with tools, and see what it cost. Every one of those paths is covered by passing automated tests.

Beneath that, the platform has something most products at this stage do not: **an internal system that measures it and refuses to flatter it.** That system currently reports that the product is not ready to launch, and it is right.

Biggest engineering achievements

- **2,595 passing tests, including the full database and queue integration surface**, verified against real infrastructure.
- **Genuine provider neutrality.** Sixteen adapters, no vendor name in business logic, and a working path requiring no vendor account at all. This is rare and it is worth real money in negotiation and in enterprise procurement.
- **A refusal to fabricate, enforced structurally.** Fabricated embeddings were found and removed rather than tuned. Unmeasured is a distinct state from zero everywhere in the system.
- **Privacy probing that has caught real leaks.** Two genuine data leaks were found by running known-sensitive values through real egress paths — including one in a field that "looked like metadata".
- **Dangerous capability removed rather than forbidden.** The automation execution layer has four operations and physically cannot merge or deploy.
- **A measured 50% CI cost reduction** at identical coverage, found by audit and verified live.

Biggest product strengths

- **Cost legibility.** Per-conversation and per-provider cost with anomaly detection is a real enterprise buying trigger, and it is built.
- **Organisational memory.** Workspaces, projects, folders, templates and audit trails mean AI work accumulates as an asset rather than evaporating into personal accounts.
- **Trustworthy failure.** The product tells the truth when something is unavailable. In a category where confident fabrication is the default failure mode, this is a differentiator that compounds.

Biggest remaining risks

Risk	Severity	Assessment
AI quality is entirely unmeasured	High	Three quality gates need a judge model. The machinery is built; it has never been run against real answers. Until then, no claim about answer quality is defensible.
Nothing has run at scale	High	No load test, no production deployment, no real concurrency. Scalability is designed for and inspectable, not demonstrated.
The capability gap between directive and repository	High	Agents, canvas, plugins, marketplace, SDK, mobile, desktop, offline and sync do not exist. If any is externally committed, that commitment is currently unbacked.

Risk	Severity	Assessment
CI is down	Medium	An infrastructure and billing condition, not a code defect — but it means no independent verification of recent work. Mitigated for this document by running everything locally.
Schema exceeds surface	Medium	121 models is a large maintenance surface, and the feedback, evaluation and model-registry clusters are richer in schema than in product.
Disaster recovery unrehearsed	Medium	Documented, never drilled. A restore procedure that has not been executed is a hypothesis.
Single-maintainer concentration	Medium	The engineering record, conventions and rationale live substantially in one place. The blueprint and this document exist partly to reduce that exposure.

Launch readiness

Not ready, and the platform says so itself: one of ten gates measured. That is not a setback — it is the system working as designed. The measured gate (privacy) passes at 100% across two declared egress paths. The other nine are unmeasured, and unmeasured is honestly reported as neither pass nor fail.

The critical path to a defensible launch decision is short and specific: wire a local model as the evaluation judge (no credentials, no cost), import public benchmark datasets, run adversarial and fault-injection suites, restore CI, and build out the end-to-end suite. None of that requires funding or production infrastructure.

Enterprise readiness

Enterprise requirement	State
Multi-tenancy with backend-enforced isolation	Implemented
Audit logging across sensitive operations	Implemented
Encrypted secret handling with audited reads	Implemented
Role-based permissions and admin separation	Implemented
Data privacy controls, consent and GDPR erasure	Implemented
Vendor independence	Implemented — unusually strong
SSO / SAML / SCIM	Not implemented
SOC 2 / ISO 27001 evidence	Not started
Accessibility conformance statement	Not audited
Contractual uptime and support processes	Not established
Penetration test	Not performed

Investment readiness

The engineering asset is real and inspectable: roughly 78,000 lines of tested source, 390 commits, a coherent architecture, and — most unusually — a self-measuring quality system with a documented record of catching its own defects.

For diligence purposes the honest framing is this: **Synora is a strong platform-engineering asset with unproven AI quality and no operating history.** Both gaps are addressable without capital, using local models and free public benchmarks, and closing them is the highest-value work available. Presenting the platform before those gates are measured would risk exactly the kind of overstatement the platform was built to avoid.

PART XI

Investor view

Differentiation and defensibility, argued from implemented architecture — with the unproven parts kept visibly separate.

> **How to read this part**
>
> Every claim below is labelled **[Implemented]** where it rests on code verified in this audit, or **[Planned]** / **[Unproven]** where it does not. Market sizing and financial projections are deliberately absent: no market research exists in this repository, and inventing figures would violate the standard the rest of this document is held to.

Market opportunity

[Not established in this repository.] No market sizing, competitive analysis, or customer research exists in the codebase or its documentation. Any figure presented here would be invented. What can be stated factually is the problem set Synora addresses — scattered AI work, vendor lock-in, invisible spend, and unmeasurable quality — each of which is a documented pain in organisations adopting AI, and each of which has implemented machinery in the product.

Product and technical differentiation

Differentiator	Substance	Basis
Structural provider neutrality	16 adapters behind one interface; no vendor named in business logic; a working path with no vendor account. Switching providers is configuration, not a rewrite.	[Implemented]
Self-measuring platform	38-module system that probes real code paths and reports three states, refusing to average unknowns into a score.	[Implemented]
Honesty as architecture	Unavailable capabilities fail honestly; fabricated output has been actively removed where found.	[Implemented]
Cost legibility	Per-model pricing with explicit basis, credit ledger, per-conversation cost, anomaly detection.	[Implemented]
Evidence loop	Real failures become permanent regression tests; benchmarks import with content hashes and preserved holdout splits.	[Implemented, lightly exercised]
Answer quality	No measurement exists yet.	[Unproven]
Scale economics	No load test or production operating history.	[Unproven]

Platform defensibility

- **Switching costs accrue to the customer's own data** — conversations, memory, templates, tools, evaluation datasets and audit history — rather than to a proprietary model. [Implemented]
- **Provider neutrality is a durable negotiating position.** A platform that can move workloads between sixteen vendors, or onto local models, is not exposed to any single vendor's pricing. [Implemented]
- **The evaluation corpus compounds.** Captured failures and imported benchmarks accumulate into a quality asset that a new entrant cannot copy. [Implemented, small today]

- **Trust compounds in regulated buying.** Probed privacy coverage, audit trails and honest reporting are precisely what enterprise diligence tests. [Implemented; certifications not started]

Cost efficiency

The platform is architected to run at effectively zero recurring infrastructure cost until launch justifies otherwise: local Postgres and Redis, local object storage, self-hosted observability, and a local-model path for AI. **[Implemented.]** A formal cost audit exists, and its one measured finding — a doubled CI pipeline — was eliminated at no loss of coverage.

At scale, the dominant cost will be AI inference. The instrumentation to manage it — per-provider cost tracking, anomaly detection, and evidence-driven routing across quality, cost and latency — is built ahead of the spend rather than after it. **[Implemented; never exercised against real billing.]**

Enterprise value proposition

One workspace where AI work is organised and attributable **[Implemented]**, isolated per tenant and enforced server-side **[Implemented]**, priced transparently **[Implemented]**, extensible via governed tools **[Implemented, HTTP transport only]**, independent of any model vendor **[Implemented]**, and continuously measured for quality **[machinery Implemented, measurement Unproven]**. Certifications, SSO, and contractual guarantees are **[Not started]**.

Long-term platform vision

The direction is a system where an organisation's AI experience compounds: every conversation a potential template, every failure a permanent test, every model change a measured decision. The loop that makes this possible is implemented in primitive form today. Extending it — agents, canvas, marketplace, SDK, mobile — is **[Planned or Not implemented]**, and this document deliberately does not present any of it as progress.

PART XII

Appendices

Verification log, absent-capability register, and glossary.

Appendix A — Verification log

Item	Method	Result
Backend suite	Vitest against local PostgreSQL 16 + Redis 7, 27 migrations applied	2,098 / 2,098 passed
Web suite	Vitest + Testing Library	462 / 462 passed
Design system suite	Vitest	35 / 35 passed
Type checking	tsc --noEmit across workspace	8 / 8 packages clean
Linting	ESLint across workspace	0 errors
Model count	Enumerated from schema.prisma	121
Migration count	Enumerated from migrations directory	27
Route modules	Enumerated from src/routes	43
AI adapters	Enumerated from src/ai	16
EOS modules	Enumerated from src/eos	38
Design-system components	Enumerated from packages/design-system	27
Web feature components	Enumerated from apps/web/src/components	49
Source lines	Counted, excluding tests and generated output	78,352
Test lines	Counted	43,791
Commits	git rev-list --count HEAD	390
Design tokens	Read from tokens.css	Reproduced exactly in Part VI
Absent capabilities	Searched across working tree and specifications	Recorded in Appendix B
CI status	GitHub Actions API — runner_id 0, no logs, seconds-long failures	Infrastructure/billing, not code

Appendix B — Absent-capability register

Named in directives; not present in the repository. Recorded so no reader mistakes intent for progress, and so a future engineer can see exactly where the boundary was on this date.

Capability	Search performed	Finding
AI Agents	Source-wide search for agent orchestration	Only HTTP user-agent header parsing
AI Canvas	Source and specification search	No implementation, no specification
Plugin system	Source and specification search	Tool engine exists; plugin system does not
Marketplace	Source and specification search	None
Extension system	Source and specification search	None

Capability	Search performed	Finding
MCP	Source-wide search	A single enum member in a type union
Automation / workflow engine	Source and specification search	Queue workers only
Public SDK	Package and source search	None published
Mobile / desktop apps	Repository search	Responsive web only
Offline mode / sync engine	Source and specification search	None

Appendix C — Glossary

Term	Meaning
Adapter	A swap-in implementation of a vendor-neutral interface. Supplying a credential activates it with no code change.
Launch gate	A condition that must be measured and met before launch. Reports passed, failed, or unmeasured.
Unmeasured	No evidence exists. Distinct from failing, and never rendered as zero.
EOS	Engineering Operating System — the internal platform that measures Synora.
Holdout split	A reserved fraction of a benchmark never used in routine measurement, so overfitting becomes visible.
Egress path	Any route by which data leaves the system; each is probed with known sensitive values.
Credential-gated	Fully implemented, inactive until a credential is supplied.
404-not-403	Unauthorised access returns "not found", so existence is not disclosed.
p95 latency	The value 95% of requests come in under — the tail users actually feel, which a mean conceals.

Synora — Platform Overview. Snapshot at commit c2f3284, 4 August 2026. Every figure measured from the repository. Every absence stated. Nothing estimated where measurement was possible.